

From Array to Sorted Array -- The Heap's Grand Finale

Lecture 14

Data Structures and Algorithms

Part 1: Binary Heaps -- Concept and Representation

A Different Kind of Tree

Every tree we have studied so far has been about **searching** -- BSTs, AVL trees, Red-Black trees, and B-Trees are all designed to find a specific key efficiently. Binary heaps solve a completely different problem.

The question: Given a collection of elements, how do we efficiently find and remove the **largest** (or smallest) element, while still being able to insert new elements quickly?

This is the **priority queue** problem. Think of:

- A hospital emergency room, where the most critical patient is treated next
- An operating system scheduler, where the highest-priority process runs next
- A network router, where the most urgent packet is forwarded first

We briefly encountered priority queues earlier in the course, implemented using sorted or unsorted arrays. Both had a weakness: either insertion was $O(n)$ or extraction was $O(n)$. The binary heap gives us **$O(\log n)$ for both** -- and it does so with a remarkably simple structure.

What is a Binary Heap?

A **binary heap** is a **complete binary tree** that satisfies the **heap property**.

Two flavors:

Type	Heap Property	Root Contains
Max-Heap	Every parent is greater than or equal to both children	Maximum element
Min-Heap	Every parent is less than or equal to both children	Minimum element

We will focus on **max-heaps** throughout this lecture. Everything works symmetrically for min-heaps (just flip the comparisons).

What a heap is NOT:

- It is **not** a BST. There is no ordering between siblings -- only between parent and child.
- The left child is not necessarily smaller than the right child.
- You cannot do an efficient general search in a heap (no $O(\log n)$ find for arbitrary keys).

A heap answers exactly one question efficiently: "**What is the maximum (or minimum)?**" And it answers it in **$O(1)$** -- just look at the root.

Complete Binary Tree -- A Quick Reminder

A **complete binary tree** is a binary tree where:

- **Every level is fully filled**, except possibly the **last level**
- The last level is filled **from left to right**, with no gaps

Complete:	NOT complete:	NOT complete:
<pre> 10 / \ 8 7 / \ / 5 3 2</pre>	<pre> 10 / \ 8 7 / \ \ 5 3 2</pre> (gap on the left at last level)	<pre> 10 / \ 8 7 / \ / \ 5 3 2 1 \ 6</pre> (last level not left-filled)

Why complete? Because a complete binary tree has a beautiful property: it can be stored in a **flat array** with no pointers at all. The shape of the tree is fully determined by the number of elements.

Array Representation -- The Key Insight

Since a complete binary tree has no "holes," we can lay out its nodes in an array **level by level, left to right**:

Tree:

Array (0-indexed):

50

/ \

30 40

/ \ / \

10 20 35 25

Index: 0 1 2 3 4 5 6

Value: [50, 30, 40, 10, 20, 35, 25]

The magic formulas (for 0-indexed arrays):

Relationship	Formula
Parent of node at index i	$\lfloor (i - 1) / 2 \rfloor$
Left child of node at index i	$2i + 1$
Right child of node at index i	$2i + 2$

Verify: Node 30 is at index 1. Left child = $2(1)+1 =$ index 3 (value 10). Right child = $2(1)+2 =$ index 4 (value 20). Parent = $(1-1)/2 =$ index 0 (value 50). All correct.

No pointers needed. Navigation is pure arithmetic. This makes heaps extremely **memory-efficient** and **cache-friendly**.

Why Array Representation Matters

Compare the memory overhead of a heap versus a pointer-based tree:

Representation	Per-Node Storage	For 1 Million Nodes
Pointer-based tree	data + left ptr + right ptr = ~24 bytes	~24 MB
Array-based heap	data only = ~4 bytes (for integers)	~4 MB

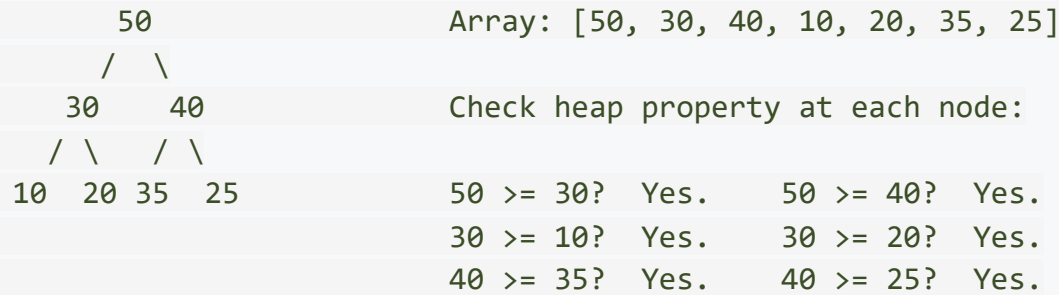
The array representation uses roughly **one-sixth** the memory. But the real advantage is **cache performance**.

Modern CPUs read memory in chunks called **cache lines** (typically 64 bytes). When you access an array element, the CPU loads nearby elements into the cache for free. With a pointer-based tree, following a pointer might jump to a completely different part of memory, causing a **cache miss** every time.

For a heap with a million elements, the array fits neatly in a few megabytes of contiguous memory. A pointer-based tree of the same size would scatter nodes across memory, with each pointer dereference potentially causing a cache miss.

This is why heaps are among the fastest data structures in practice, not just in theory.

Max-Heap: Verifying the Property



This is a valid max-heap. The maximum element (50) is at the root (index 0). Finding the maximum takes **$O(1)$** -- it is always at `array[0]`.

An important observation: The second-largest element must be one of the root's children (since only the root is greater). But the **third-largest** could be anywhere -- a child of the root, or even a grandchild. There is no guarantee about the ordering of elements at the same level or in different subtrees.

The heap property is intentionally weak. It gives up the ability to search efficiently in exchange for the ability to insert and delete-max very quickly.

Part 2: Heap Operations

Insert -- Swim Up (Bubble Up)

To insert a new element into a max-heap:

1. **Place the new element at the end** of the array (this maintains the complete tree shape)
2. **Swim up**: Compare the new element with its parent. If the new element is larger, **swap** them. Repeat until the element reaches a position where it is smaller than its parent, or it becomes the root.

Why does this work?

Adding at the end preserves the complete tree property. Swimming up restores the heap property: each swap fixes the violation at the current position and may create a new one at the parent (which the next iteration handles).

Complexity:

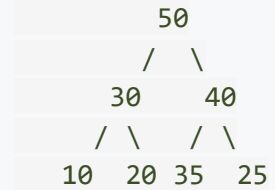
The new element can swim up at most from a leaf to the root -- that is, the **height** of the tree. For a complete binary tree with n nodes, the height is $\lfloor \log_2 n \rfloor$.

Insert: $O(\log n)$

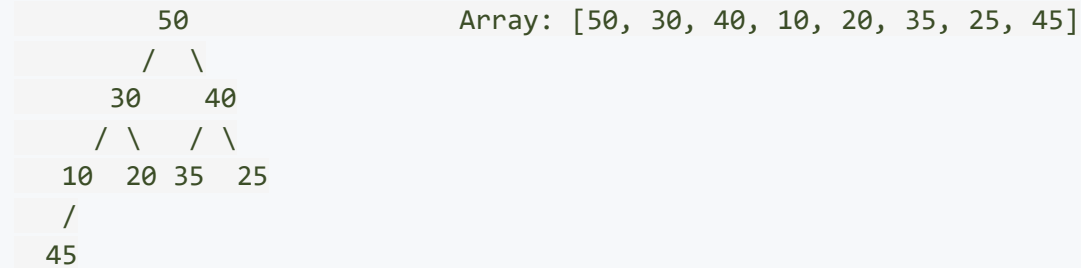
Insert: Traced Example

Insert **45** into this max-heap:

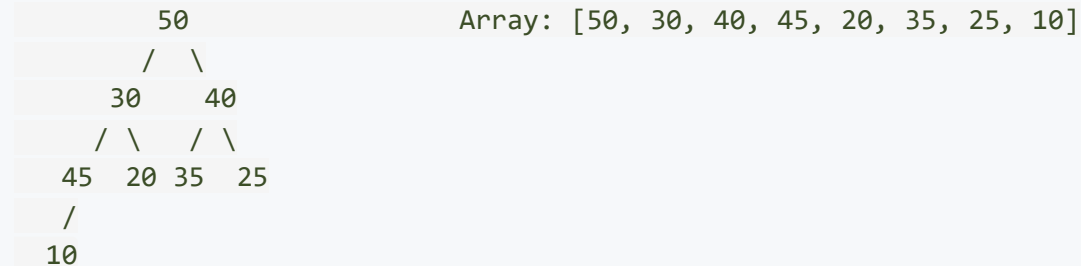
Step 0 (Before): Array: [50, 30, 40, 10, 20, 35, 25]



Step 1: Place 45 at the end (index 7).

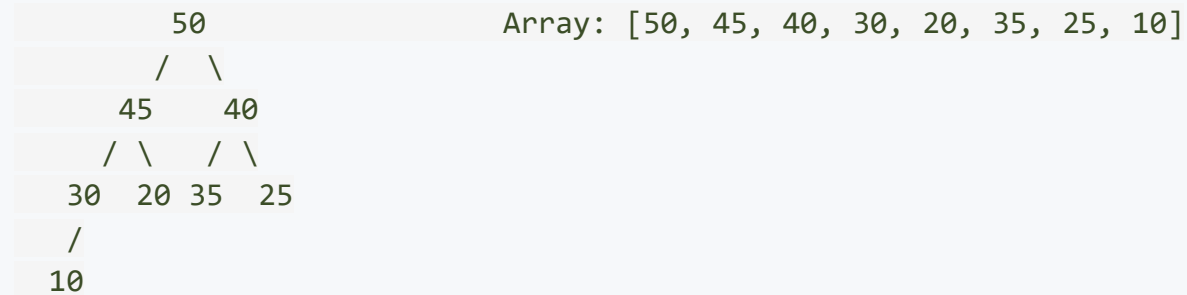


Step 2: Compare 45 with parent (10 at index 3). $45 > 10 \rightarrow$ SWAP.



Insert: Traced Example (continued)

Step 3: Compare 45 with parent (30 at index 1). $45 > 30 \rightarrow$ SWAP.



Step 4: Compare 45 with parent (50 at index 0). $45 < 50 \rightarrow$ STOP.

Done. The element 45 swam up from index 7 to index 1 in **2 swaps**. The heap property is restored everywhere. Notice that the element found its natural resting place -- above all elements smaller than it, below the one element larger than it on that path. The rest of the tree was not disturbed at all.

Insert: Code

```
void insertHeap(int heap[], int* size, int key) {  
    // Step 1: Place at the end  
    int i = *size;  
    heap[i] = key;  
    (*size)++;  
  
    // Step 2: Swim up  
    while (i > 0) {  
        int parent = (i - 1) / 2;  
        if (heap[i] > heap[parent]) {  
            // Swap with parent  
            int temp = heap[i];  
            heap[i] = heap[parent];  
            heap[parent] = temp;  
            i = parent;           // Move up  
        } else {  
            break;               // Heap property satisfied  
        }  
    }  
}
```

The loop runs at most $\lfloor \log_2 n \rfloor$ times. Each iteration does a constant amount of work (one comparison, one potential swap). **Total: $O(\log n)$.**

Delete-Max -- Sink Down (Bubble Down)

Removing the maximum element (the root) from a max-heap:

1. **Replace the root** with the **last element** in the array
2. **Reduce the heap size** by 1
3. **Sink down**: Compare the new root with its children. Swap with the **larger child** if it is bigger. Repeat until the element reaches a position where it is larger than both children, or it becomes a leaf.

Why swap with the larger child?

If we swapped with the smaller child, the larger child would be below a smaller parent, violating the heap property. Swapping with the larger child guarantees the new parent is at least as large as both children.

Complexity:

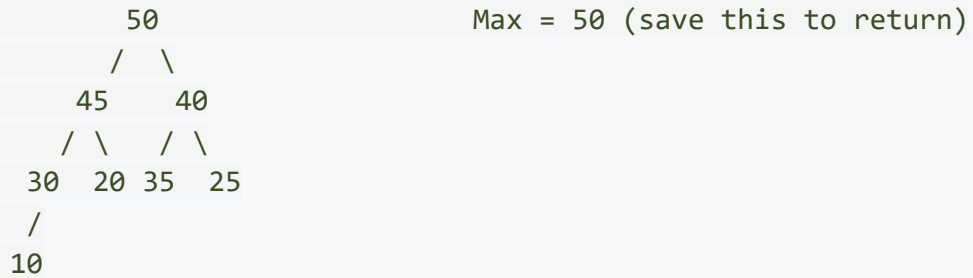
The element can sink at most from root to leaf = $\lfloor \log_2 n \rfloor$ levels.

Delete-Max: $O(\log n)$

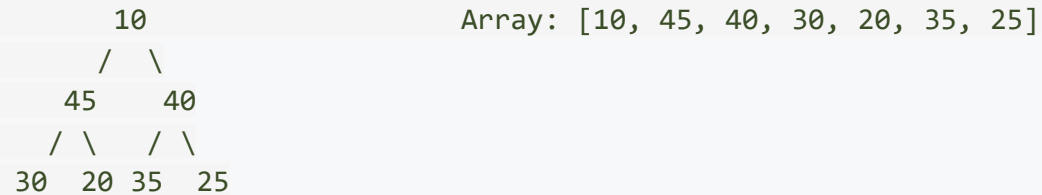
Delete-Max: Traced Example

Delete the maximum from this max-heap:

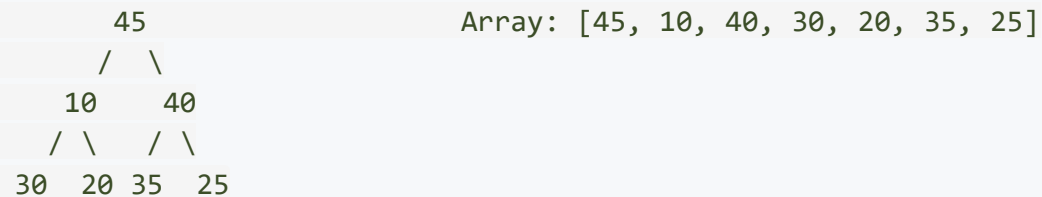
Step 0 (Before): Array: [50, 45, 40, 30, 20, 35, 25, 10]



Step 1: Replace root with last element (10). Reduce size.

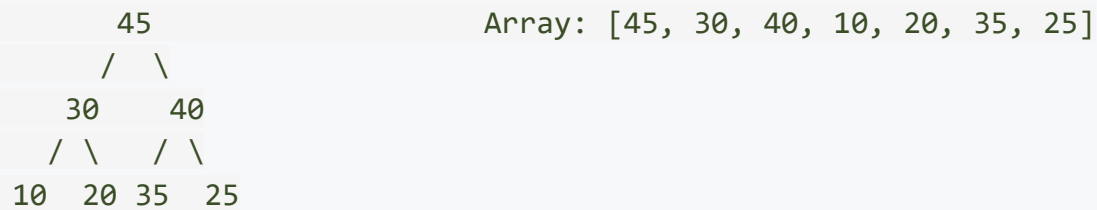


Step 2: Sink down. Compare 10 with children (45, 40).
Larger child is 45. $10 < 45 \rightarrow$ SWAP.



Delete-Max: Traced Example (continued)

Step 3: Compare 10 with children (30, 20).
Larger child is 30. $10 < 30 \rightarrow$ SWAP.



Step 4: Node 10 is now at index 3. Its only child would be at index 7, which is beyond the heap size. 10 is a leaf $\rightarrow$ STOP.

Done. The new root (10, taken from the end) sank down 2 levels in 2 swaps. The heap property is fully restored. The maximum (50) has been removed and returned.

The shape of the tree is still complete -- we always remove the last element in the array and place it at the root, then fix the ordering. This guarantees the array stays compact.

Delete-Max: Code

```
int deleteMax(int heap[], int* size) {
    if (*size == 0) return -1;    // Heap is empty

    int max = heap[0];            // Save the maximum
    (*size)--;
    heap[0] = heap[*size];        // Move last element to root

    // Sink down
    int i = 0;
    while (1) {
        int left = 2 * i + 1;
        int right = 2 * i + 2;
        int largest = i;

        if (left < *size && heap[left] > heap[largest])
            largest = left;
        if (right < *size && heap[right] > heap[largest])
            largest = right;

        if (largest != i) {
            int temp = heap[i];    // Swap
            heap[i] = heap[largest];
            heap[largest] = temp;
            i = largest;           // Move down
        } else {
            break;                // Heap property satisfied
        }
    }
    return max;
}
```

The Heapify Function

The "sink down" operation is so important that it gets its own name: **heapify** (also called **sift-down** or **percolate-down**). It is the building block for everything that follows.

```
void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;
    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;
        heapify(arr, n, largest);    // Recurse down
    }
}
```

heapify(arr, n, i) assumes that the subtrees rooted at the children of **i** are already valid heaps. It fixes the violation at node **i** by sinking it to the correct position. This is exactly what delete-max does after placing the last element at the root.

Heap Operations -- Summary So Far

Operation	Algorithm	Time	Space
Find Max	Return <code>arr[0]</code>	$O(1)$	$O(1)$
Insert	Add at end, swim up	$O(\log n)$	$O(1)$
Delete Max	Move last to root, sink down (heapify)	$O(\log n)$	$O(1)$

Compare this with our earlier implementations of priority queues:

Implementation	Insert	Delete Max	Find Max
Unsorted array	$O(1)$	$O(n)$	$O(n)$
Sorted array	$O(n)$	$O(1)$	$O(1)$
Binary Heap	$O(\log n)$	$O(\log n)$	$O(1)$

The heap strikes the best balance. Neither insert nor delete-max is as fast as the best case of either array approach, but neither is ever as slow as the worst case. This is the hallmark of a well-designed data structure -- **no operation is cheap at the expense of making another operation expensive.**

Part 3: Build-Heap -- Floyd's Algorithm

The Problem: Convert an Arbitrary Array into a Heap

Given an arbitrary array of n elements, we want to rearrange it so that it satisfies the max-heap property.

The naive approach:

Start with an empty heap. Insert elements one at a time.

- Each insertion costs $O(\log n)$
- n insertions cost $O(n \log n)$

This works, but can we do better?

The answer is yes -- and the improvement is surprising.

Floyd's Build-Heap algorithm converts any array into a valid heap in $O(n)$ time. Not $O(n \log n)$. Just $O(n)$. This is one of those rare and satisfying results in computer science where the clever approach is not just a constant factor better, but an entire logarithmic factor better.

Floyd's Build-Heap: The Idea

The key observation is brilliantly simple:

Leaves are already valid heaps. A node with no children trivially satisfies the heap property -- there is nothing to violate.

In a complete binary tree with n nodes, the leaves occupy indices $\lfloor n/2 \rfloor$ through $n - 1$. That means **roughly half the array is already a valid heap** without any work.

Floyd's algorithm:

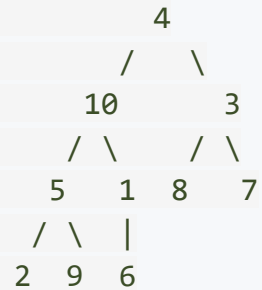
1. Start at the **last non-leaf node** (index $\lfloor n/2 \rfloor - 1$)
2. Apply **heapify** (sink down) at that node
3. Move to the previous node, apply heapify again
4. Continue until you reach the root (index 0)

By processing bottom-up, when we heapify node **i**, both its children's subtrees are already valid heaps (because we already heapified all nodes below it). This is exactly the precondition that heapify requires.

Build-Heap: Traced Example

Convert the array **[4, 10, 3, 5, 1, 8, 7, 2, 9, 6]** into a max-heap.

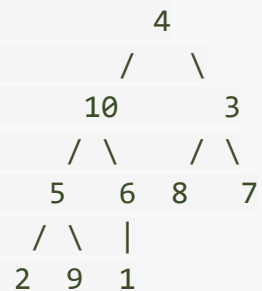
Initial tree (just place the array into tree form):



$n = 10$, so last non-leaf = $\text{floor}(10/2) - 1 = \text{index } 4$ (value 1)

Step 1: Heapify at index 4 (value 1). Children: index 9 (value 6).

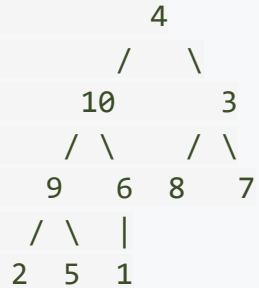
- $1 < 6 \rightarrow$ swap 1 and 6.



Build-Heap: Traced Example (continued)

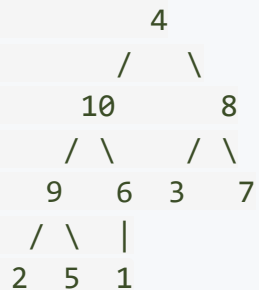
Step 2: Heapify at index 3 (value 5). Children: 2 (index 7) and 9 (index 8).

- Largest child is 9. $5 < 9 \rightarrow$ swap 5 and 9.



Step 3: Heapify at index 2 (value 3). Children: 8 (index 5) and 7 (index 6).

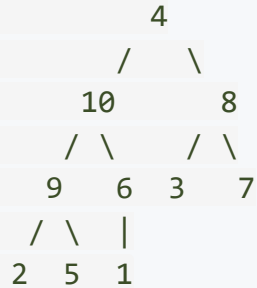
- Largest child is 8. $3 < 8 \rightarrow$ swap 3 and 8.



Build-Heap: Traced Example (continued)

Step 4: Heapify at index 1 (value 10). Children: 9 (index 3) and 6 (index 4).

- $10 > 9$ and $10 > 6$ --> no swap needed. Already valid.



Step 5: Heapify at index 0 (value 4). Children: 10 (index 1) and 8 (index 2).

- Largest child is 10. $4 < 10$ --> swap 4 and 10.



Build-Heap: Traced Example (final)

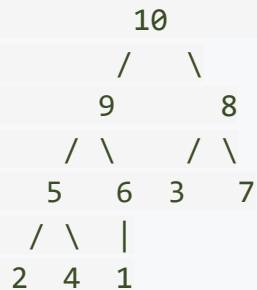
After swapping 4 and 10, node 4 (now at index 1) must continue sinking. Children: 9 (index 3) and 6 (index 4).

- Largest child is 9. $4 < 9 \rightarrow$ swap 4 and 9.



Continue: node 4 (now at index 3). Children: 2 (index 7) and 5 (index 8).

- Largest child is 5. $4 < 5 \rightarrow$ swap 4 and 5.



Final array: [10, 9, 8, 5, 6, 3, 7, 2, 4, 1]

Done. The array is now a valid max-heap. Every parent is greater than or equal to both children. Verify it yourself.

Why Build-Heap is $O(n)$, Not $O(n \log n)$

This is the part that surprises most people. We called heapify $n/2$ times, and heapify can take $O(\log n)$ at worst. So should the total be $O(n/2 * \log n) = O(n \log n)$?

No. The key is that most nodes do very little work.

In a complete binary tree with n nodes:

- **$n/2$ nodes** are leaves -- they need **0 swaps** (we skip them entirely)
- **$n/4$ nodes** are one level above the leaves -- they need at most **1 swap**
- **$n/8$ nodes** need at most **2 swaps**
- ...and so on...
- **1 node** (the root) needs at most **$\log n$ swaps**

The total work is:

$$\sum_{k=0}^{\log n} \frac{n}{2^{k+1}} \cdot k = n \sum_{k=0}^{\log n} \frac{k}{2^{k+1}}$$

The sum $\sum_{k=0}^{\infty} k/2^{k+1}$ converges to **1**. Therefore the total work is bounded by **n** .

The intuition: Most nodes are near the bottom and sink very little. Only a few nodes are near the top and sink far. The "heavy" work at the top is overwhelmed by the "light" work at the bottom.

Part 4: Heap Sort

Heap Sort: The Idea

Heap Sort is a beautiful application of the binary heap. The idea is simple:

1. **Build a max-heap** from the input array -- $O(n)$ using Floyd's algorithm
2. **Repeatedly extract the maximum** and place it at the end of the array:
 - Swap `arr[0]` (the current max) with the last element of the unsorted portion
 - Reduce the heap size by 1
 - Call heapify on the root to restore the heap property
 - Repeat until the heap has one element

After step 2, the array is sorted in **ascending order** because the largest elements were moved to the end first, then the next-largest, and so on.

Why is this clever?

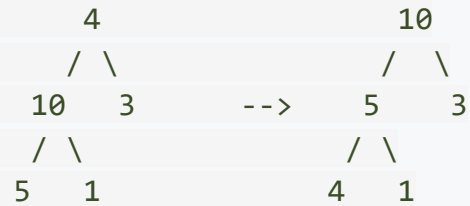
We are using the heap's own array to store the sorted output. As the heap shrinks from the left, the sorted portion grows from the right. **No extra array is needed** -- the sort is **in-place**.

Heap Sort: Traced Example

Sort the array **[4, 10, 3, 5, 1]** using Heap Sort.

Step 1: Build max-heap

Initial: After build-heap:

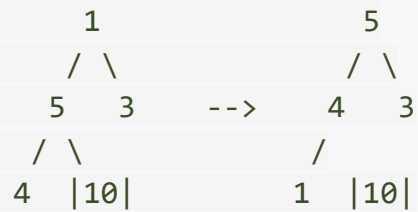


Array: [4, 10, 3, 5, 1] --> [10, 5, 3, 4, 1]

Step 2: Extract maximum repeatedly

Iteration 1: Swap arr[0]=10 with arr[4]=1. Heap size becomes 4. Heapify root.

Swap: Heapify: Sorted portion: [10]

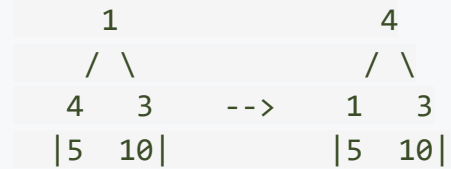


Array: [1, 5, 3, 4, |10|] --> [5, 4, 3, 1, |10|]

Heap Sort: Traced Example (continued)

Iteration 2: Swap $\text{arr}[0]=5$ with $\text{arr}[3]=1$. Heap size becomes 3. Heapify root.

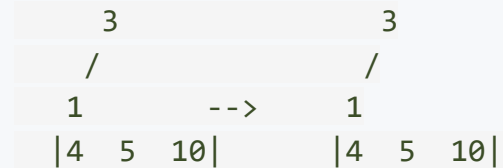
Swap: Heapify: Sorted portion: [5, 10]



Array: [1, 4, 3, |5, 10|] --> [4, 1, 3, |5, 10|]

Iteration 3: Swap $\text{arr}[0]=4$ with $\text{arr}[2]=3$. Heap size becomes 2. Heapify root.

Swap: Heapify: Sorted portion: [4, 5, 10]



Array: [3, 1, |4, 5, 10|] --> [3, 1, |4, 5, 10|] (already valid heap)

Heap Sort: Traced Example (final)

Iteration 4: Swap $\text{arr}[0]=3$ with $\text{arr}[1]=1$. Heap size becomes 1. Done.

Array: $[1, |3, 4, 5, 10|]$ $-->$ $[1, 3, 4, 5, 10]$

The array is sorted: $[1, 3, 4, 5, 10]$.

Complexity breakdown:

Phase	Work
Build max-heap	$O(n)$
$n-1$ extract-max operations, each with heapify	$n-1$ calls to heapify, each $O(\log n)$
Total	$O(n) + O(n \log n) = O(n \log n)$

Heap Sort: Code

```
void heapSort(int arr[], int n) {  
    // Step 1: Build max-heap (Floyd's bottom-up)  
    for (int i = n / 2 - 1; i >= 0; i--)  
        heapify(arr, n, i);  
  
    // Step 2: Extract elements one by one  
    for (int i = n - 1; i > 0; i--) {  
        // Swap root (max) with last unsorted element  
        int temp = arr[0];  
        arr[0] = arr[i];  
        arr[i] = temp;  
  
        // Heapify the reduced heap  
        heapify(arr, i, 0);  
    }  
}
```

That is the entire algorithm: just **two loops**. The heapify function (shown earlier) does all the heavy lifting. The first loop builds the heap bottom-up. The second loop repeatedly extracts the max and places it at the correct sorted position.

Notice: We are not using any extra array. The original array is partitioned into two regions -- the heap on the left and the sorted output on the right. As the heap shrinks, the sorted region grows.

Heap Sort: Properties

Property	Value
Best-case time	$O(n \log n)$
Average-case time	$O(n \log n)$
Worst-case time	$O(n \log n)$
Space	$O(1)$ -- in-place
Stable?	No

What makes Heap Sort special:

Guaranteed $O(n \log n)$ in the worst case with $O(1)$ extra space. No other comparison-based sort achieves both.

- Quick Sort is usually faster in practice but degrades to $O(n^2)$ in the worst case
- Merge Sort is always $O(n \log n)$ but requires $O(n)$ extra space
- Heap Sort is always $O(n \log n)$ and uses only $O(1)$ extra space

What holds it back:

Heap Sort has poor **cache locality**. The heapify operation jumps between parent and child nodes, which may be far apart in the array for large heaps. This makes it slower than Quick Sort in practice (typically 2-3x slower), despite having a better worst-case guarantee.

Sorting Algorithms -- The Grand Comparison

Algorithm	Best	Average	Worst	Space	Stable?	Notes
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	Guaranteed, in-place
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	Stable, needs extra space
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Fastest in practice
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Best for small/nearly sorted
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Always slow
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Educational only

When to use Heap Sort:

- When you need a **guaranteed worst-case** $O(n \log n)$ sort
- When **extra memory is not available** (embedded systems, kernel code)
- When you need a **partial sort** -- find the k largest elements in $O(n + k \log n)$
- As a **fallback** for Quick Sort's worst case (this combination is called Introsort)

Part 5: The Trees Unit -- Complete Review

The Tree Family -- Our Journey

Trees Unit (Lectures 9-14):

Binary Tree (Lectures 9-10)

- Terminology, types, properties
- Traversals: in-order, pre-order, post-order, level-order
- Threaded Binary Trees: $O(1)$ space traversal

|

Binary Search Tree (Lecture 11)

- The power of ordering: search, insert, delete
- The skew problem: sorted input produces $O(n)$ height

|

Balanced BSTs (Lecture 12)

- AVL Tree: balance factor, four rotation cases, insertion, deletion
- Red-Black Tree: five color properties, relaxed balance, industry standard

|

Multi-Way Trees and B-Trees (Lecture 13)

- The disk access problem: why binary trees fail on disk
- B-Tree: wide nodes, perfect balance, $O(\log_m n)$ height
- B+ Trees: database indexing, linked leaves for range queries

|

Binary Heaps and Heap Sort (Lecture 14 -- today)

- Array-based complete binary tree
- Insert (swim), Delete-max (sink), Build-Heap $O(n)$
- Heap Sort: $O(n \log n)$, in-place, guaranteed

Key Complexity Summary -- All Tree Structures

Structure	Search	Insert	Delete	Space	Best For
BST (balanced)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$	General in-memory
BST (worst case)	$O(n)$	$O(n)$	$O(n)$	$O(n)$	-- avoid this --
AVL Tree	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$	Search-heavy workloads
Red-Black Tree	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$	Insert/delete-heavy
B-Tree (order m)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$	Disk-based data
Binary Heap	$O(n)$	$O(\log n)$	$O(\log n)$	$O(n)$	Priority queues, sorting

Notice the trade-offs:

- AVL and Red-Black trees both guarantee $O(\log n)$ for everything, but differ in how strictly they enforce balance
- B-Trees sacrifice nothing on complexity, but are designed for a fundamentally different cost model (disk accesses)
- Heaps are the odd one out: they cannot efficiently search, but they are the only structure that supports $O(1)$ find-max and $O(n)$ construction

Choosing the Right Tree Structure

"I need to..."	Best Choice	Why
Search frequently, rarely insert/delete	AVL Tree	Shortest tree, fastest search
Insert/delete frequently in memory	Red-Black Tree	Bounded rotations, quick maintenance
Store data on disk, minimize disk reads	B-Tree / B+ Tree	Wide nodes, low height
Support range queries on disk	B+ Tree	Linked leaves enable sequential scan
Find the min or max quickly	Binary Heap	$O(1)$ find-max, $O(\log n)$ operations
Sort in-place with worst-case guarantee	Heap Sort	$O(n \log n)$ always, $O(1)$ space
General-purpose sorted collection	Red-Black Tree	Good all-around performance
Build once, search many times	AVL Tree	Optimal search after construction

Every data structure we have studied is the best answer to at least one real-world question. The skill is not in knowing the algorithms, but in recognizing **which problem you are actually solving**.

Looking Ahead

This lecture concludes our study of **tree data structures**. But trees will not disappear from the course -- they will return in unexpected places.

In the next unit (Graphs):

- Trees are a special case of graphs -- a connected, acyclic graph
- **DFS** on a graph is really just pre-order traversal generalized to allow cycles
- **BFS** on a graph is level-order traversal generalized to allow cycles
- **Spanning trees** extract a tree from a graph, and we use heaps to build minimum spanning trees efficiently (**Prim's algorithm**)
- **Shortest path algorithms** (Dijkstra's) use a min-heap as their core data structure

The patterns you learned in the trees unit -- recursive traversal, balanced restructuring, heap operations -- are foundational techniques that appear throughout the rest of computer science.

Key Takeaways

What We Covered Today

1. Binary Heaps -- Concept

- A complete binary tree satisfying the heap property (max-heap or min-heap)
- Stored as a flat array -- no pointers, no wasted space
- Parent, left child, right child computed by simple arithmetic on array indices

2. Heap Operations

- Insert: add at end, swim up -- $O(\log n)$
- Delete-max: replace root with last, sink down (heapify) -- $O(\log n)$
- Find-max: just look at `arr[0]` -- $O(1)$

3. Build-Heap (Floyd's Algorithm)

- Process nodes bottom-up, from last non-leaf to root
- Each node is heapified assuming its children's subtrees are already valid heaps
- Total work is $O(n)$, not $O(n \log n)$ -- most nodes are near the bottom and sink very little

4. Heap Sort

- Build max-heap + repeatedly extract max = sorted array
- $O(n \log n)$ time, $O(1)$ space, guaranteed worst case
- In-place but not stable, slower than Quick Sort in practice due to cache behavior

Review Questions

1. Given the array **[15, 10, 20, 8, 12, 25, 5]**, draw the corresponding complete binary tree. Is this a valid max-heap? If not, which nodes violate the heap property?
2. Insert the value **18** into the max-heap **[20, 15, 10, 8, 12, 5, 3]**. Show each swap during the swim-up process. What is the final array?
3. Delete the maximum from the max-heap **[50, 30, 40, 10, 20, 35, 25]**. Show each swap during the sink-down process. What value is returned and what is the final array?
4. Apply Floyd's Build-Heap algorithm to the array **[3, 7, 1, 9, 5, 2, 8]**. Show the tree after heapifying each non-leaf node, starting from the last non-leaf.
5. Explain why Build-Heap runs in $O(n)$ time, even though heapify takes $O(\log n)$ in the worst case. What makes the total work less than $O(n \log n)$?
6. Sort the array **[6, 2, 8, 1, 4, 3]** using Heap Sort. Show the heap and sorted portion after each extraction.
7. Why is Heap Sort not stable? Give an example of two equal elements whose relative order changes during the sort.
8. A real-time system needs to always process the highest-priority task and cannot tolerate worst-case $O(n)$ operations. Which data structure should it use for its task queue, and why?